WNDRVR
SUPPORT NETWORK

VXWORKS 7 CUSTOM COMPONENT AND CDF DEVELOPER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Custom Component and CDF Developer's Guide, SR0650

10 June 2020

1. UNDERSTANDING VXWORKS COMPONENTS AND CDFS

1. VxWorks Components and CDFs

A VxWorks® component is a basic unit of functionality with which VxWorks can be configured using Workbench or the vxprj command-line facility.

VxWorks components are defined in component description files (CDFs) using the VxWorks component description language (CDL). This language provides a syntax for defining component objects and their properties (such as their code modules and initialization routines). It also provides syntax for defining objects for configuration parameters, initialization groups, component bundles and component profiles, as well as for Workbench folders and selections.

In effect, CDFs provide an external interface with Workbench and vxprj that allows users to select and configure components, and an internal interface with the underlying configuration and build facilities that defines the code used to generate a VxWorks image.

2. VxWorks Components, CDFs and Layers

VxWorks components and their descriptions are always contained within a more fundamental VxWorks construct called a layer.

VxWorks is composed of layers, each containing technologies made up of components. All operating system facilities must be implemented as components within layers. Components and their CDF file descriptions must always be encapsulated by layers. Having created a custom VxWorks layer for your custom operating system facility (such as a file system or networking protocol), you populate it with components comprised of both CDFs and implementation code files.

In addition to being used for operating system facilities, layer encapsulated CDL and CDFs are also used for drivers and BSPs. For information about these topics, see the *VxWorks 7 BSP and Driver Guide*.

Layers allow you to manage your custom facilities in the same way as you manage standard Wind River supplied VxWorks features from Workbench, wrtool and vxprj. For further information about layers and how to manage them, see the *VxWorks 7 Layers and Package Management Guide*.

Note: VxWorks functionality can be added in the form of kernel modules that do not have to be defined as VxWorks components. However, Wind River recommends using layer-encapsulated CDFs when developing VxWorks operating system extensions. In order to use Workbench, wrtool or the vxprj facilities with custom code, operating system extensions must be developed as layers containing components comprised of both CDFs and implementation source files.

3. Post SR0600 VxWorks File Paths

VxWorks Standard Release 0600 (SR0600) updated the VxWorks layer architecture, and the Wind River product installation file paths.

While the implementation of components and layers has not been changed by SR0600, the overall structure of the Wind River supplied layers has been modified.

The SR0600 restructuring work has resulted in a change to the product installation directory path. Wind River supplied layers are now found in *installDir* **/vxworks-7/pkgs_v2**.

If you are using Wind River VxWorks SR0600 and onwards, you must substitute this product installation directory path to all of the pre-SR0600 VxWorks layer and component examples specified in this document. Where you encounter a reference to *installDir* **/vxworks-7/pkgs**, replace this path with *installDir* **/vxworks-7/pkgs_v2**.

4. Post SR0600 VxWorks Helix File Paths

VxWorks Standard Release 0600 (SR0600) and following changes how the VxWorks guest installation is arranged.

While VxWorks helix technologies are still composed of layers and components, their organization in the Wind River product installation directory has changed.

If you are using the Wind River SR0600 release onwards, then the following file path adjustments must be made to all of the pre-SR0600 VxWorks layer and component examples specified in this document.

The helix hypervisor and ARINC 653 safety system implementations are located in *installDir* **/helix/pkgs_v2**. If you are developing for VxWorks, you will not be required to make any CDF related changes to files in these subdirectories.

If you are developing for the VxWorks, you will locate VxWorks related CDF files in the directory *installDir* **/helix/guests/vxworks-7/pkgs_v2**. Where you encounter a reference to *installDir* **/vxworks-7/pkgs** or *installDir* **/vxworks-7/pkgs_v2** in the examples that follow, replace these paths with *installDir* **/helix/guests/vxworks-7/pkgs_v2**.

2. CREATING A CUSTOM COMPONENT

1. Code Development for Components

Code for operating system facilities is always provided in components that are encapsulated by layers.

Multiple components are usually implemented within a single VxWorks layer.

When you decide to create a custom component, you must locate that component in a VxWorks layer. You must therefore decide whether or not to append your component to an established layer, or create a new layer for it. Wind River recommends you do not append custom components to Wind River supplied layers. Rather, you must encapsulate all new components within new application layers, or append them to existing ones. For instructions on how to create new layers, refer to the *VxWorks 7 Layers and Package Management Guide*.

The advantage of implementing your components within layers is they are automatically read by the project build system. This means your layers appear for selection in the VxWorks source build configuration tool, or are selectable from the command-line interface (CLI) vxprj command. Your components therefore appear in the VxWorks image project configuration tool and are also actionable from the CLI.

Component initialization modules may be implemented within a component layer by either binary or source code. For information about the CDF properties that define component initialization, see Component Initialization Properties on page 19 and Initialization Group Object and Properties on page 28.

Component code that makes use of exposed parameters used by the project facilities (Workbench or vxprj) must be provided as source code, rather than binary code.

For information about how the CDF properties define component parameters and associated code, see:

- Source File Properties on page 20.
- Configuration Parameter Property on page 21 (for the component object).
- Parameter Object and Properties on page 25.

For a description of how components are represented in the VxWorks layer architecture, see Code Installation Directories on page 3.

2. Code Installation Directories

Wind River delivers the various elements of its VxWorks components in the following layer directories:

- The *layerDir* **/src** directory contains component library source files, where *layerDir* is the layer's root directory.
- Component header files are found in *layerDir* **/h**. For the related component property, see *HDR_FILES* in Source File Properties on page 20.
- Component object modules and object module archives are usually found in the *layerDir* **/precomp** directory and are part of binary layers, where some or all of the layer's functionality is implemented in object code and no source code is provided. For the related component properties, see *LINK_SYMS* and *MODULES* in Module Linkage Properties on page 16. For an explanation of how to create and manage binary layers, see the VxWorks 7 Layers and Package Management Guide.
- Component description files are usually installed in *layerDir* **/cdf**. Alternatively, for VxWorks Board Support Package (BSP) layers and CPU architecture library layers, they are installed in the layer's root directory *layerDir* . For information about the contents of CDFs, see Object Class Overview on page 13.
- Component configlettes are component source code functions, or source fragments, and are located in *layerDir* **/configlette**. For the related component property, see *CONFIGLETTES* in Source File Properties on page 20.

Layers are either located internally to the Wind River installation, below the *installDir* **/vxworks-7/pkgs_v2** directory, or externally from it. For further details about locating layer directories externally from the Wind River installation directory, see *VxWorks 7 Outside Layers Tutorial*.

The advantage of implementing components in VxWorks layers is they are automatically read by the project facility.

WNDRVR

3. INTEGRATING A CUSTOM COMPONENT

1. Component Definitions and CDF File Names

Components are defined in their CDFs, and the syntax *nnSomeName* **.cdf** is used for naming CDF files, where *nn* is a two-digit number.

Once a custom component has been implemented, you must integrate it into VxWorks either by creating a new CDF file containing the various component attributes, or by appending your custom component definition to a pre-existing CDF file. One CDF file is permitted to define multiple components and their attributes, which are all encapsulated by the same VxWorks layer.

Name a new CDF file that defines component attributes using the following syntax: **00comp_foo.cdf**, where the name of the new component is foo. The **.cdf** file suffix is required. Wind River recommends the decimal prefix as a convenient way to control the order in which CDF files are read within a directory by the project build system.

CDF files in layer directories are read by the project build system in alphanumeric order. For more details, refer to CDF Processing in Layers on page 6.

You must ensure your CDF file name is unique, and so differs from all other CDF files across all the other layers used by your Wind River installation. Your VxWorks Image Project (VIP) will not build correctly if a CDF file name clash exists between two different components in different layers included into VxWorks at the same time.

> Note: If two layers are mutually exclusive, they will not both be included as part of the VxWorks source build project at the same time, and so it is acceptable for these layers to include CDF files with identical file names.

> Note: Wind River reserves the first 50 numbers, that is **00someName.cdf** through **49someName.cdf**. The remaining numbers, 50 through 99, may be used by third parties for their components.

2. CDF Files in Layer Directories

When you create a new component in a new CDF file, you will usually locate this file in the component's layer directory.

The majority of component CDF files are located in the Wind River installation within VxWorks layer directories. Usually layers are located in the Wind River installation directory at *installDir* **/vxworks-7/pkgs_v2**.

Layers can also be located externally to the Wind River installation by appending a directory path to the workstation environment variable *WIND_LAYER_PATHS*. The build system locates any valid VxWorks layers positioned at the specified directory paths. Refer to *VxWorks 7 Outside Layers Tutorial* for more details.

> Note: In a Helix Virtualization Platform installation, there is a second environment variable named *WIND_LAYER_PATHS_PLATFORM*. Supply custom layers to the Helix installation by appending their directory paths to this environment variable. The *WIND_LAYER_PATHS* environment variable co-exists as the way to supply custom layers to the native VxWorks install.

Two layer types exist, and each one arranges CDF files differently.

- Layer type 1: Board Support Package (BSP) and CPU architecture library layers.
  CDF files are contained in the layer root directory. For example, *installDir* **/vxworks-7/pkgs_v2/os/board/intel/itl_generic**.
- Layer type 2: All other VxWorks layers.
  CDF files are contained beneath the layer's root directory in **/cdf**. For example, *installDir* **/vxworks-7/pkgs_v2/storage/fs/util**.

3. CDF Processing in Layers

More than one CDF may define a given component and its properties within the component's layer.

CDF files are processed at VxWorks Image Project (VIP) project configuration time. At this point, the CDFs from all the source build (VSB) project layers have already been copied from the VxWorks layer directories into the VSB project *projDir* **/krnl/cdf** subdirectory. When the VIP is configured, the build system reads these copied CDF files within the VSB in alphanumeric order.

The order in which CDFs are read is significant. If more than one CDF describes the same property of the same component, the last one read by the build system overrides and amends all earlier ones in the VIP. CDFs read later therefore have precedence over those that are read earlier. If the later CDF doesn't specify the same set of fields, then the older CDF file fields are combined with those in the newer CDF files. The precedence of multiple definitions is determined by a numbering scheme used with the CDF file naming convention. The build system reads the CDF files in alphanumeric order and gives precidence to the file names with higher value.

For example, if the Board Support Package (BSP) layer defines two component files named **00bsp.cdf** and **20bsp.cdf**, then **20bsp.cdf** has higher precedence. The build system reads any BSP definition that appears in both files, and it combines the definition from **20bsp.cdf** in the VIP configuration with the definition from **00bsp.cdf**.

4. Wind River Reserved CDF File Names

Wind River reserves eight CDF file names for particular uses by the project build system.

When using these particular CDF file names, make sure the contents matches the description given below. Do not use these file names for purposes other than those described below.

> Note: Wind River has also reserved object name prefixes for the various component object types that are contained in the CDF files described below. For further information, refer to Reserved Object Name Prefixes Reference on page 13.

In each file name, *nn* are the recommended two-digit prefixes. A single component CDF can define more than one component and attribute. The Wind River reserved CDF file names are:

- *nn* **bsp.cdf**, **bsp.cdf**, **bsp653.cdf**, **bspcert.cdf**

  Located normally within the root directory of a board support package (BSP) layer, these files contain board-specific definition and configuration parameters such as the kernel entry point address in memory, and configuration parameters for board specific components.
- *nn* **comp_compName.cdf**

  This file contains one or more component and related parameter definitions. For simple components, the definitions are sometimes joined by folder definitions, which are related to the components defined here. The text *compName* in the file name must be substituted with the name of the primary component defined in this file.
- *nn* **folder_folderName.cdf**

  This file is often used alongside other CDF files within a VxWorks layer containing a large number of individual components. The folder is presented by both the Workbench project configuration tool and the **vxprj** command-line build tool. All the components named in this CDF file are contained within the folder. The text *folderName* in the file name must be substituted with the name of the folder defined in this file.
- *nn* **bundle_bundleName.cdf**

  You define a bundle in this file as a group of related components that are defined in other CDF files. Bundles are useful because they ensure unresolved dependencies do not occur in the configuration of a VxWorks project. The text *bundleName* in the file name must be substituted with the name of the bundle defined in this file.
- *nn* **profile_profileName.cdf**

  You define a profile in this file as a component oriented baseline of VxWorks operating system functionality, which modifies the default component configuration available with the VxWorks product installation. The text *profileName* in the file name must be substituted with the name of the profile defined in this file.

5. Overriding Kernel Component Definitions

The recommended approach for overriding kernel component definitions is the creation of a new layer, although other approaches to modifying kernel components are also available.

Where possible, you establish an update to a kernel component in a new copy of the component's layer directory. Overriding component properties therefore involves overriding VxWorks layer versions as well.

Because you are creating a new layer version, the option to choose between layer versions in subsequent VxWorks Source Build (VSB) projects becomes available. This robust approach affords flexibility, while retaining the option to revert to the original Wind River layer and component if required. Refer to *VxWorks 7 Layers and Package Management Guide* for details.

Sometimes you will need to make minor modifications to pre-existing VxWorks component definitions in the CDF file. In this case, you need not create a new layer. Instead, you can modify the kernel component by creating a higher precedence CDF file outside the layer which overrides the component's original definition.

Overriding Component Properties from the Board Support Package Layer on page 7CDF files located in the Board Support Package (BSP) layer will override related CDFs in subsequently created VxWorks Source Build (VSB) projects and VxWorks Image Projects (VIP).
Example of Using the BSP Layer to Override Component Properties on page 7 This example modifies the **INCLUDE_DISK_UTIL** component for all VxWorks Image Projects (VIP) based on the VxWorks simulator BSP layer.
Overriding Component Properties from the VxWorks Image Project on page 8CDF files located in the VxWorks Image Project (VIP) will override related CDFs assembled in the VxWorks Source Build (VSB) project.
Example of Using the VxWorks Image Project to Override Component Properties on page 8 This example modifies the **INCLUDE_DISK_UTIL** component for a particular VxWorks Image Project (VIP) in the workspace.
Overriding Component Properties from Non-Layer Directories on page 9 You can override a component definition for all VxWorks projects in your workspace without also having to override the entire layer that contains the component.
Example of Overriding Wind River Component Properties from a Non-Layer Directory on page 9 This example modifies the **INCLUDE_DISK_UTIL** component for all the VxWorks Image Projects (VIP) in the workspace.

5.1. Overriding Component Properties from the Board Support Package Layer

CDF files located in the Board Support Package (BSP) layer will override related CDFs in subsequently created VxWorks Source Build (VSB) projects and VxWorks Image Projects (VIP).

This approach to component modification lets you change the CDF without overriding the whole component layer. The BSP layer directory stores a copy of the original component CDF file that contains the minor component redefinitions. The project build system reads this new CDF file from the BSP layer directory at project creation time and interprets it as a higher precedence CDF file. The component definition is therefore overridden, and the behaviour of the component is changed in the project.

For this method to work, you must first establish which BSP layer you intend to base your VxWorks projects upon. You must also locate the CDF file for the VxWorks component that will be modified. Once you have incremented the numeric portion of the CDF file name and copied the file to the BSP layer directory, then you will make your component modifications in that new CDF file.

Once the BSP layer contains the modified CDF, VIP creation with this BSP causes the project build system to import the modified CDF file along with the BSP layer and your component modifications take effect for the VIP.

Parent topic: Overriding Kernel Component Definitions on page 7

5.2. Example of Using the BSP Layer to Override Component Properties

This example modifies the **INCLUDE_DISK_UTIL** component for all VxWorks Image Projects (VIP) based on the VxWorks simulator BSP layer.

About This Task

You will modify the component's SYNOPSIS element, which is used by the VIP configuration tool to display a descriptive message about the component. This example assumes the component **INCLUDE_DISK_UTIL** is included in the VIP. Only newly created VIPs that are based on the VxWorks simulator BSP will inherit this overridden component.

Procedure

1. Locate the component CDF file in *installDir* **/vxworks-7/pkgs_v2/storage/fs/util/cdf/10comp_fsutil.cdf**.
2. Copy this CDF file to *installDir* **/vxworks-7/pkgs_v2/os/board/wrs/vxsim/20comp_fsutil.cdf**.
3. Edit **20comp_fsutil.cdf** and change the SYNOPSIS element.

```
Component INCLUDE_DISK_UTIL
    {
    NAME            File System and Disk Utilities
    SYNOPSIS        This component is overridden in the BSP
    MODULES         usrFsLib.o
```

4. Create a new VIP from Workbench.
5. Open the VIP and select the kernel configuration tool.
6. Locate the VxWorks component **INCLUDE_DISK_UTIL** and observe the modification made to the SYNOPSIS element. Every VIP based on the VxWorks simulator BSP from now on that includes the **INCLUDE_DISK_UTIL** component will reflect this modification.

Parent topic: Overriding Kernel Component Definitions on page 7

5.3. Overriding Component Properties from the VxWorks Image Project

CDF files located in the VxWorks Image Project (VIP) will override related CDFs assembled in the VxWorks Source Build (VSB) project.

If your VIP already exists, and you need to modify a VxWorks kernel component within it quickly, then instead of copying the CDF file from the component layer to the Board Support Package (BSP) layer, instead you can copy it to the VIP project directory. You then apply the modifications to the CDF file in the VIP directory. You can copy the CDF file either to the VIP project root directory, or the sub-directory for the project's BSP layer. The component modifications are applied the next time you open the VIP in Workbench.

The benefit of this approach is that it lets you modify a kernel component quickly without requiring the creation of a new VIP. The downside of this approach is that, the change is restricted to one VIP and if you eventually delete it, you will also delete the modifying CDF file and so lose the overriding component changes.

Parent topic: Overriding Kernel Component Definitions on page 7

5.4. Example of Using the VxWorks Image Project to Override Component Properties

This example modifies the **INCLUDE_DISK_UTIL** component for a particular VxWorks Image Project (VIP) in the workspace.

About This Task

You will modify the component's SYNOPSIS element, which is used by the VIP configuration tool to display a descriptive message about the component. This example assumes the component **INCLUDE_DISK_UTIL** is included in a VIP in the workspace called **myVIP**. Only **myVIP** will inherit this overridden VxWorks component.

WNDRVR

Procedure

1. Locate the component CDF file in *installDir* **/vxworks-7/pkgs_v2/storage/fs/util/cdf/10comp_fsutil.cdf**.
2. Copy this CDF file to *installDir* **/workspace/myVIP/20comp_fsutil.cdf**.
3. Edit **20comp_fsutil.cdf** and change the SYNOPSIS element.

```
Component INCLUDE_DISK_UTIL
    {
    NAME            File System and Disk Utilities
    SYNOPSIS        This component is overridden in the VIP
    MODULES         usrFsLib.o
```

4. Locate the project **myVIP** in the workspace and open the project in Workbench.
5. Open the kernel configuration tool and locate the component **INCLUDE_DISK_UTIL**.
6. Observe the modification made to the SYNOPSIS element.

Parent topic: Overriding Kernel Component Definitions on page 7

5.5. Overriding Component Properties from Non-Layer Directories

You can override a component definition for all VxWorks projects in your workspace without also having to override the entire layer that contains the component.

You use this approach by selecting the CDF file to override the kernel component, then copying that CDF files to a non-layer directory on the filesystem. Create the environment variable *WIND_USER_CDF_PATH* on the workstation, and assign the directory file system path to it. The project build system will parse the CDF files in those directories when it opens a VIP in the workspace.

This approach allows you to store CDF files that override VxWorks components in non-layer application source code directories instead of VxWorks layer directories.

WIND_USER_CDF_PATH is carried over from previous releases of VxWorks, and so it maintains a level of backwards compatibility with those earlier VxWorks releases.

Parent topic: Overriding Kernel Component Definitions on page 7

5.6. Example of Overriding Wind River Component Properties from a Non-Layer Directory

This example modifies the **INCLUDE_DISK_UTIL** component for all the VxWorks Image Projects (VIP) in the workspace.

About This Task

You will modify the component's SYNOPSIS element, which is used by the VIP configuration tool to display a descriptive message about the component. This example assumes the component **INCLUDE_DISK_UTIL** is included in the VIP, that a Windows workstation is in use and that a directory called **C:\cdf** exists in the filesystem.

Procedure

1. Locate the component CDF file in *installDir* **\vxworks-7\pkgs_v2\storage\fs\util\cdf\10comp_fsutil.cdf**.
2. Copy this CDF file to **C:\cdf\20comp_fsutil.cdf**.
3. Create the environment variable *WIND_USR_CDF_PATH* and assign the directory path **C:\cdf** to it.
4. Edit **C:\cdf\20comp_fsutil.cdf** and change the SYNOPSIS element.

```
Component INCLUDE_DISK_UTIL
                        {
                        NAME            File System and Disk Utilities
                        SYNOPSIS        Overridden using WIND_USR_CDF_PATH
                        MODULES         usrFsLib.o
```

5. Create a new VIP in Workbench.
6. Open the VIP and select the kernel configuration tool.
7. Locate the VxWorks component **INCLUDE_DISK_UTIL** and observe the modification made to the SYNOPSIS element. Every newly created VIP that includes the **INCLUDE_DISK_UTIL** layer will reflect this modification.

Parent topic: Overriding Kernel Component Definitions on page 7

6. CDF Directory Precedence

CDF files can be used to override VxWorks kernel components, and there is a precedence enforced between the directories that can be used to store those CDF files.

The CDF files that are assembled into the VxWorks Source Build (VSB) project directory *projDir* **/krnl/cdf** are processed by the build system in file name alphanumeric order.

However, CDF files that modify VxWorks components will also be located in other locations on the workstation file system. For example, the VxWorks Image Project (VIP) will also contain Board Support Package (BSP) related CDF files as well.

There is a precedence applied to CDF files by the project build system based on the location of the directory where the CDF files are stored. CDFs in higher precedence directories will always override CDFs in lower precedence directories, whatever the alphanumeric comparison between the CDF file name turns out to be.

The CDF directory precedence order that is enforced by the project build system for CDF directories is as follows:

Table 1. CDF Directory Precedence

Precedence Level	Directory Path
Highest Precedence	
	VIP subdirectory for BSP layer, for example *installDir* **/workspace/myVIP/itl_generic**.
	VIP project root directory, for example *installDir* **/workspace/myVIP**.
	Directories pointed to by *WIND_USER_CDF_PATH* .
	VSB CDF subdirectory, e.g. *installDir* **/workspace/myVSB/krnl/cdf**.
Lowest Precedence	

This means that if the CDF **10comp_fsutil.cdf** is found in the VSB directory *installDir* **/workspace/myVSB/krnl/cdf**, then whatever numeric value is assigned to a second CDF file name, *nn* **comp_fsutil.cdf**, if it is located in one of the other related directories then it will always have a higher precedence and so override this component's definition.

7. CDF Precedence in VxWorks 7 Compared to VxWorks 6.9

While the use of CDF files has not changed between VxWorks product releases, the rules for CDF directory precedence have changed.

VxWorks CDF file name precedence rules remain alphanumeric, but because the VxWorks directory structure has changed, the CDF directory precedence rules have changed to account for the directory change.

Table 1. CDF Precedence in VxWorks 6.9 Compared With VxWorks 7

CDF Precedence in VxWorks 6.9	CDF Precedence in VxWorks 7
CDF file precedence is alphanumeric within a directory. **CDF directory precedence overrides CDF file name precedence.** CDF files reside in various different VxWorks installation directories: *installDir* **/vxworks-6.9/target/config/comps/VxWorks** *installDir* **/vxworks-6.9/target/config/comps/VxWorks/arch** *installDir* **/vxworks-6.9/target/config/comps/VxWorks/tool** *installDir* **/vxworks-6.9/target/config/** *bspName* **Host environment variable** *WIND_USER_CDF_PATH* **points to host workstation application directories that also contain CDF files.**	**CDF file precedence is alphanumeric within a directory.** **CDF directory precedence overrides CDF file name precedence. Refer to** CDF Directory Precedence on page 10 **for more information.** CDF files reside in various different VxWorks installation directories: **VxWorks Source Build (VSB) project directories contain copies of layer CDFs. For example,** *installDir* **/workspace/myVSB/krnl/cdf.** **VxWorks Image Project (VIP) directories contain copies of Board Support Package (BSP) layer CDFs. For example,** *installDir* **/workspace/myVIP.** **Host environment variable** *WIND_USER_CDF_PATH* **points to host workstation application directories that also contain CDF files.**

8. CDF Testing

There are several tests that can run to verify that components have been written correctly.

Check Syntax, Semantics, and Availability

This vxprj command provides the most basic test:

```
vxprj component check [projectFile] [component ... ]
```

For example:

```
% vxprj component check MyProject.wpj
```

If no project file is specified, vxprj looks for a **.wpj** file in the current directory. It checks if a component is valid. If no component is specified, it checks every component in the project. Also, it checks if a component is unavailable (if a module is missing) or if any of the required components are unavailable. Note that unavailable components are greyed out in the Workbench configuration facility.

Based on test output, make the required modifications. Keep running the script until you have removed the errors.

Check Component Dependencies

The simplest way to check dependencies is to add and remove the component from a test project (using Workbench or vxprj), and check which components are added and removed. The vxprj component dependencies command can be used for further analysis, if required, using the following syntax:

```
vxprj component dependencies [projectFile] component [component ... ]
```

For example, the following command displays a list of components required by **INCLUDE_OBJ_LIB**, as well as those that require the component:

```
% vxprj component dependencies INCLUDE_OBJ_LIB
```

If no project file is specified, vxprj looks for a **.wpj** file in the current directory.

Check the Component Hierarchy in Workbench

Verify that selections, folders, and new components you have added are properly included by making a visual check of the Workbench component hierarchy.

Look at how your new elements appear in the folder tree. Check the parameters associated with a component and their parameter default values.

If you have added a folder containing components, and have included that folder in your configuration, the Workbench component hierarchy will display in boldface all components listed as defaults for that folder (that is, values for the **DEFAULTS** property).

Check prjConfig.c

Inspect the project file **prjConfig.c** to make sure that the initialization routine is generated properly and in the correct order. For information about **prjConfig.c** and other project files, see About CDFs, VxWorks Configuration, and Build on page 12.

Build and Boot the System

Verify that the resulting image builds and boots.

Note: The Workbench kernel configuration facility does not automatically register changes made to CDFs after a VxWorks image project (VIP) has been created. The VIP must be closed and re-opened for the changes to be accessible through Workbench. The vxprj facility reads CDFs each time it is invoked.

9. About CDFs, VxWorks Configuration, and Build

The information provided in CDF files is used by the configuration facilities (underlying both the Workbench and vxprj interfaces) to dynamically generate C files in the project directory that are used by the build system.

These files must not be modified:

- **linkSyms.c**
- **prjComps.h**
- **prjConfig.c**
- **prjParams.h**
- **Makefile.mk**

4. CDF OBJECTS AND PROPERTIES

1. Object Class Overview

CDFs may use several classes of objects, each of which has various properties. There is no mandatory order for information within a CDF file—with regard to either the order of objects in relationship to one another, or the order of properties within an object.

The object classes are as follows.

Operating System Objects

The following sections describe the objects used to create a component for an operating system facility or device driver:

- Component Object and Properties on page 14
- Parameter Object and Properties on page 25
- Initialization Group Object and Properties on page 28

Component Grouping Objects

The following sections describe the objects used to group components into a single unit (bundle or profile) for ease of configuration:

- Bundle Object and Properties on page 31
- Profile Object and Properties on page 33

Workbench Objects

The following sections describe the objects used to control the display and selection of components in the Workbench kernel configuration interface:

- Workbench Folder Object and Properties on page 36
- Workbench Selection Object and Properties on page 38

BSP Objects

The following section describes the object used to define a BSP:

- BSP Object and Properties on page 40

2. Reserved Object Name Prefixes Reference

Several prefixes are reserved for naming specific object types.

INCLUDE_

for components.

BUNDLE_

for component bundles.

PROFILE_

for profiles.

FOLDER_

for Workbench folders.

SELECT_

for Workbench selections.

The prefixes are recommended to avoid naming conflicts, but not required. Do not use these prefixes for object types other than the ones for which they are reserved.

3. Component Object and Properties

Component objects define units of operating system functionality or device drivers so that they can be configured into (and out of) a system using Workbench or the vxprj command-line project facility.

The simplest component requires a component name and identification of the modules associated with it. Others require identification of initialization routines and their places in initialization groups, parameters, their dependencies on other components, and display properties in the Workbench component configuration interface. The properties of the component object are used to accomplish these requirements.

New components must be independent of existing CDF objects, with two exceptions:

- New component object initialization routines must be associated with an existing initialization group object.
- New component objects must be bound into an existing folder or selection object for Workbench display and selection purposes.

More than one component may be defined in a single CDF file. It is useful to do so when a set of components form a logical group. For example, in all VxWorks releases from SR0600 onwards refer to the CDFs in the VxWorks kernel **core** and **debug_analysis** layers because both contain various components. You can locate those layers in *installDir* **/vxworks-7/pkgs_v2/os/core** and *installDir* **/vxworks-7/pkgs_v2/os/debug_analysis**.

For information about displaying component information in Workbench kernel configuration interface, see Workbench Folder Object and Properties on page 36 and Workbench Selection Object and Properties on page 38.

For information about how to group components into component bundles and profiles, see Bundle Object and Properties on page 31 and Profile Object and Properties on page 33

For information about file naming conventions, see Component Definitions and CDF File Names on page 5.

For detailed information about creating components and CDFs for device drivers, see the *VxWorks 7 BSP and Driver Guide*.

Component Template

```
Component          INCLUDE_MY_COMP {
    NAME           user-friendly name for GUI
    SYNOPSIS       description for GUI
    CONFIGLETTES   file1.c file2.c . . .
    MODULES        aLib.o bLib.o . . .
    INIT_RTN       initMyComp(MAX_THIS MAX_THAT) ;
```

```
    HDR_FILES      header1.h header2.h . . .
    LINK_SYMS      aFuncName bFuncName . . .
    CFG_PARAMS     MAX_THIS  \
                   MAX_THAT
                    . . .
    REQUIRES       INCLUDE_FOO1  \
                   INCLUDE_FOO2  \
                   INCLUDE_FOO3  \
                    . . .
    ARCHIVE        /somePrivateDir/myArchiveDir/myArchive.a
    INCLUDE_WHEN   INCLUDE_BAR1  \
                   INCLUDE_BAR2  \
                   INCLUDE_BAR3  \
                    . . .
    _INIT_ORDER    myInitGroup
    INIT_BEFORE    INCLUDE_FOOBAR
    _CHILDREN      FOLDER_MY_FOLDER
    _DEFAULTS      FOLDER_MY_FOLDER
}
```

Component Name on page 15A component must have a unique name. The name is used by both Workbench and vxprj to identify the component.
Component Properties on page 16 Component properties are assigned values to define attributes of the component.
Parameter Object and Properties on page 25 Parameter objects are used to define component configuration parameters that are presented to the user for modification by way of Workbench or vxprj.
Initialization Group Object and Properties on page 28 If a component has an initialization routine, it must be identified as such in the component object (using the component **INIT_RTN** attribute), and it must also be registered with an initialization group.
Bundle Object and Properties on page 31 Bundle objects can be used to associate components that are often used together, which facilitates configuration of the operating system with useful sets of facilities.
Profile Object and Properties on page 33A profile object defines a set of components to be used as a starting point to create a project.
CDL Underscore Operator and Object Relationships on page 35 The underscore operator can be used with object properties that define a relationship to reverse the definition of that relationship—usually along the lines of changing list-of-members to member-of.
Workbench Folder Object and Properties on page 36 Folder objects control the display of components in the Workbench kernel configuration facility.
Workbench Selection Object and Properties on page 38A selection object is used to display alternative options for components that provide related functionality (such as timestamp drivers) in the Workbench component configuration interface.
BSP Object and Properties on page 40BSP objects define BSPs and their properties so that they can be selected for a VxWorks image project (VIP) using Workbench or the vxprj command-line project facility.
BSP Name on page 40A BSP must have a unique name. The BSP name is used by both Workbench and vxprj to identify the BSP.
BSP Properties on page 41
BSP Example on page 42 Review code for a BSP object.

3.1. Component Name

A component must have a unique name. The name is used by both Workbench and vxprj to identify the component.

For operating system facilities, the name must use an **INCLUDE_** prefix to avoid naming conflicts. For example, **INCLUDE_HRFS**.

For device driver components, particularly VxBus-compliant drivers, the name must use the following syntax:

```
DRV_driverType _driverName
```


WNDRVR

For example, **DRV_SIO_NS16550**.

The name must also be valid as a C macro name, as it is used as such by the VxWorks configuration facilities.

The syntax for component-name assignment is as follows, using the Component keyword and name before the brackets that enclose the component-object properties:

```
Component INCLUDE_MYCOMP {
// component property assignments go here
}
```

Also see Component Object and Properties on page 14 and Component Example on page 24 for the context in which the name assignment is made.

The component name appears in the **Name** column in the Workbench display of components (see Workbench Display Properties on page 23). Note that the component name is not the same as the more user-friendly descriptive string used with the **NAME** attribute.

Parent topic: Component Object and Properties on page 14

3.2. Component Properties

Component properties are assigned values to define attributes of the component.

For example:

- Object code (modules) and source code that the component provides.
- Source code that is compiled by the project (sometimes referred to as configlettes because it is identified with the **CONFIGLETTES** property).
- Header files.
- Initialization routines called at system startup.
- Dependencies on other components.
- Workbench GUI display information.

Module Linkage Properties on page 16
Component Example on page 24 As an example, a message logging component is defined in CDL.

Parent topic: Component Object and Properties on page 14

Module Linkage Properties

LINK_SYMS

Provides a mechanism for linking a component's object modules into the system image when the component is added to the VxWorks kernel image (VIP) project—even when there are no symbolic references to those modules. This is accomplished by assigning one of the object's public symbols to **LINK_SYMS**; only one per module is needed.

Assume, for example, that foo( ) and bar( ) are provided by **fooLib.o** and **barLib.o** respectively. Then the following assignment would ensure that **fooLib.o** and **barLib.o** are linked with the VxWorks image if the component is added to the project:

```
LINK_SYMS foo bar
```

In general, all of the component's modules (except one identified with **INIT_RTN**) must be identified with **LINK_SYMS**. If an initialization routine is identified with **INIT_RTN**, it is automatically linked with the system image when the component is included; it does not need to be identified with **LINK_SYMS** as well. See *INIT_RTN* in Source File Properties on page 20.

WNDRVR

Note that **LINK_SYMS** does not define inter-component relationship or dependency. All symbols defined as **LINK_SYMS** must be provided by the component itself. For information about explicitly defining component dependencies, see Component Dependency Properties on page 22.

MODULES

Identifies the component's library object files to ensure that they are treated as a unit by the project facility when one or more of them is referenced symbolically (by another component module or application code), and that the component is registered with the project. All of the component's modules must be identified with **MODULES**.

For example, the Network File System (NFS) layer is composed of the following components:

- **INCLUDE_NFS2_CLIENT**
- **INCLUDE_NFS3_CLIENT**
- **INCLUDE_CORE_NFS_CLIENT**
- **INCLUDE_NFS2_SERVER**
- **INCLUDE_NFS3_SERVER**
- **INCLUDE_CORE_NFS_SERVER**
- **INCLUDE_NFS_DEBUG**

Each of these component definitions has its own **MODULES** component property in the NFS layer CDF, located at **nfs/cdf/01comp_nfs.cdf**. The component **INCLUDE_NFS2_CLIENT** defines two object modules:

```
Component INCLUDE_NFS2_CLIENT
    {
    NAME            NFS v2 client
    ...
    MODULES         nfs2Lib.o \
                    nfs2Drv.o
```

All the non-configlette code functions that comprise **INCLUDE_NFS2_CLIENT** are implemented within these two library object modules, and because they are defined as component modules, they are treated as a unit by the project facility. They are just two of the many library object files that are built by the NFS layer makefile.

The NFS layer makefile is located at **nfs/src/Makefile**, and it builds all of the non-configlette NFS library source files, producing library object files which are then used by the definitions of all of the components listed above, including **INCLUDE_NFS2_CLIENT**. The NFS layer makefile builds the following library object files. Notice that both **nfs2Lib.o** and **nfs2Drv.o** appear in the list of library object files that are built.

```
# Makefile - Makefile for Network File System
...
# Object files
OBJS = nfsd.o nfsHash.o nfs2dLib.o nfs3dLib.o xdr_nfs2_xdr.o \
       xdr_nfs3_xdr.o xdr_nfs_common_xdr.o xdr_mnt1_xdr.o xdr_mnt3_xdr.o \
       xdr_mnt_common_xdr.o mountd.o nfsCommon.o mount3Lib.o \
       mount1Lib.o nlm4Lib.o xdr_nlm4.o nfs3Lib.o nfs3Drv.o \
       nfs2Drv.o nfs2Lib.o nfsCommon.o nfsDebug.o
```

The component **MODULES** property ensures any object file, which is linked through a symbolic reference from another component or application code, is linked with the system image. In addition, the initialization routine (if defined) and any component requirements (see *REQUIRES* in Component Dependency Properties on page 22) are added to the system, and the component is added to the project's list of components. Thus the component is treated as a coherent unit.

If a components' modules are not identified with **MODULES**, they are linked instead through symbolic references, but the component is not added to the VIP list of components and the component initialization routine (if one is defined) is not linked.

WNDRVR

Note: Identification of modules with **MODULES** does not give an absolute guarantee that these library object files will always be linked with the system image when the component is included in the project. By default, objects that are not required to complete the final image linkage (that is, those not symbolically referenced in the code) are not linked into the VxWorks image. In order to force linkage of modules (when there are no symbolic references in the code), they must either be identified using **INIT_RTN** (for the initialization routine) or **LINK_SYMS**. See *INIT_RTN* in Source File Properties on page 20 and *LINK_SYMS* in Parameter Name on page 26.

Note: An object file must only be associated with one component, and its name must be globally unique (across the entire system).

Locate Component Object Modules in a Project Archive File on page 18 The **ARCHIVE** property identifies the archive file within the VxWorks source build (VSB) where the build system locates component object modules.
Component Initialization Properties on page 19 If a component has an initialization routine, it must be identified as such in the component object using the component **INIT_RTN** attribute, and it must also be registered with an initialization group.
Source File Properties on page 20 Source file properties include **CONFIGLETTES** and **HDR_FILES**.
Configuration Parameter Property on page 21 The **CFG_PARAMS** identifies configuration parameters associated with the component, typically a list of preprocessor macros that are exposed to users for alteration through Workbench or vxprj.
Component Dependency Properties on page 22 Dependencies between components are resolved automatically by the project facility when they are based on symbolic references.
Workbench Display Properties on page 23 The **NAME**, **SYNOPSIS**, **_CHILDREN**, and **_DEFAULTS** properties control the display of information about a component in Workbench.

Parent topic: Component Properties on page 16

Related information
Component Example on page 24

Locate Component Object Modules in a Project Archive File

The **ARCHIVE** property identifies the archive file within the VxWorks source build (VSB) where the build system locates component object modules.

When a VSB is built, some of the included layers are compiled and the resulting object files are stored within an archive file contained beneath the VSB project **krnl** build directory. When the build system compiles a subsequent kernel image project (VIP) that is based upon this VSB, the build system must be told where to locate the component object files that were archived within the VSB.

All components that were archived during the VSB build must have the **ARCHIVE** property added to their component definition. The **ARCHIVE** property must be assigned the name of the VSB project component archive file, not the name of the component that was archived there. You may need to check the VSB project directory to identify the correct archive file name. The **ARCHIVE** property is unnecessary for components encapsulated by layers that contain binary code stored within archive files.

The **ARCHIVE** property does not control whether or not the object modules in the VSB archive is linked with the system image at project build time. Rather, the property indicates to the build system during VIP build time where archived component binary object modules are located.

For example, the component **INCLUDE_SPINLOCK** is included into a VIP supporting symmetric multiprocessing (SMP). This component is defined in the VxWorks kernel layer directory. In all VxWorks pre-SR0600 releases, the file path is *installDir* **/vxworks-7/pkgs/os/core/kernel-***a.b.c.d* **/cdf/00vxWorks.cdf**, where *a.b.c.d* is the kernel layer version number. The kernel component CDF file **00vxWorks.cdf** includes the following line:

```
ARCHIVE libspl.a
```

The archive file **libspl.a** is not located in the VxWorks kernel layer directory. Rather, it is created whenever a VSB is built with symmetric multiprocessing enabled in the kernel layer and the file is located beneath the VSB **krnl** directory. For example, assuming a **NEHALEM** CPU based VSB, the archive file path is *projectDir* **/krnl/NEHALEM/common/libspl.a**.

Parent topic: Module Linkage Properties on page 16

Component Initialization Properties

If a component has an initialization routine, it must be identified as such in the component object using the component **INIT_RTN** attribute, and it must also be registered with an initialization group.

There are two alternatives for registering a component with an initialization group, and for controlling the order in which it is initialized within that group:

- From within the component object, use the **_INIT_ORDER** property to register the component with an initialization group, and (optionally) **INIT_BEFORE** to control the order. Use this method to add a component to an existing initialization group without modifying the object for that group. For more information, see *_INIT_ORDER*, *INIT_BEFORE*, and *INIT_AFTER* in Parameter Name on page 26.
- From within an initialization group object, use the **INIT_ORDER** property to both register the component with the group and to control the order of initialization. Use this method if you want to create a new initialization group for a group of custom components. For more information, see *INIT_ORDER*, *INIT_BEFORE*, and *INIT_AFTER*, in Initialization Group Properties on page 30.

The order in which the initialization groups themselves are processed is controlled by properties of initialization group objects (for more information, see Initialization Group Objects on page 29).

INIT_RTN

If the component has an initialization routine, **INIT_RTN** must be used to identify it. The assignment of the initialization routine to **INIT_RTN** has the effect of linking the module that provides the initialization routine with the VxWorks image (if the component is included in the project).

This component object must be a valid line of C source code, so must always have a semicolon at the end. While it is often used to make a function call, it could also be used to assign a value to a global variable instead.

In order to have the initialization routine called automatically at system start-up, you must also add the component to an initialization group. For information about the options available, see Parameter Name on page 26.

The initialization routine assignment is made as follows:

```
INIT_RTN fooInit(arg1, arg2);
```

If it fits on one line, the entire initialization routine itself can be assigned in **INIT_RTN**; if not, the signature must be assigned, and the code can be provided in the configelette, as a binary module in the component's archive, or as library source code (and identified by **CONFIGLETTES**; see *CONFIGLETTES* in Source File Properties on page 20). An initialization routine can also use component parameters (see *CFG_PARAMS* in Configuration Parameter Property on page 21) as arguments if it is provided in source code. Header files for initialization routines are identified with the **HDR_FILES** property (see *HDR_FILES* in Source File Properties on page 20). For information about defining the initialization order of the component, see *INIT_BEFORE* and *_INIT_ORDER* in Parameter Name on page 26.

Note that the initialization routine identified for a component object is not the same as the initialization routine for an initialization group object; the component **INIT_RTN** property is not the same as the initialization group **INIT_RTN** property.

If a component does not have an initialization routine, and any of the component's modules must be included in the system image regardless of whether or not there are external references to them, then **LINK_SYMS** must be used to identify those modules. See *LINK_SYMS* in Module Linkage Properties on page 16.

WNDRVR

_INIT_ORDER

Assigns the component to the specified initialization group. This property is used to add the component to an existing initialization group. The alternative is to add the component to the initialization group object itself (see Parameter Name on page 26 and Initialization Group Object and Properties on page 28; also see CDL Underscore Operator and Object Relationships on page 35).

In most cases, custom components must be initialized after VxWorks components, which is specified as follows:

```
_INIT_ORDER usrRoot
```

A component (or initialization group) that is assigned to an existing initialization group using the **_INIT_ORDER** property is, by default, initialized last within that group. The **INIT_BEFORE** property can be used to change the order (see *INIT_BEFORE* and *INIT_AFTER* in Parameter Name on page 26).

INIT_BEFORE

Calls the initialization routine of this component before the one specified with the **INIT_BEFORE** property. This property can be used to specify precisely where the component is placed in the initialization sequence of the initialization group specified by **_INIT_ORDER** (see *_INIT_ORDER* in Parameter Name on page 26). **INIT_BEFORE** effectively causes the component to be inserted into an existing initialization group's ordered list of components (see *INIT_ORDER* in Initialization Group Properties on page 30).

In most cases, custom components are initialized after VxWorks components; however, you can override this behavior by explicitly using the **INIT_BEFORE** property. For example:

```
...
_INIT_ORDER usrRoot
INIT_BEFORE INCLUDE_USER_APPL
...
```

In this example, the component is declared a member of the **usrRoot** initialization group (the basic VxWorks initialization group), and it will be initialized before the VxWorks component **INCLUDE_USER_APPL**.

The same mechanism can be used to control the initialization order within custom initialization groups.

The **INIT_BEFORE** property only affects ordering within the component's initialization group. Do not reference a component that is not in the initialization group; it has no effect. In addition, this property is effective only in conjunction with **_INIT_ORDER**.

INIT_AFTER

Calls the initialization routine of this component after the one specified with the **INIT_AFTER** property. This property can be used to specify precisely where the component is placed in the initialization sequence of the initialization group specified by **_INIT_ORDER** (see *_INIT_ORDER* in Parameter Name on page 26).

The **INIT_AFTER** property only affects ordering within the component's initialization group. Do not reference a component that is not in the initialization group; it has no effect. In addition, this property is effective only in conjunction with **_INIT_ORDER**.

Parent topic: Module Linkage Properties on page 16

Source File Properties

Source file properties include **CONFIGLETTES** and **HDR_FILES**.

CONFIGLETTES

Identifies C source files to be compiled as part of the component. Configlette definitions may include the use of macros. For example:

```
CONFIGLETTES $(MY_VAR)\myFile.
```

Where *MY_VAR* may either be a build macro, or an environment variable.

Typically, the **CONFIGLETTES** property is used to identify the code used for component parameters, or code conditional on the presence of a given component. Component parameters are used, for example, to specify a number, a binary setting, and so on. To be accessible to users for configuration with Workbench or vxprj, the parameters themselves must be specified with the component **CFG_PARAMS** property and a parameter object. For example, if the code identified with **CONFIGLETTES** uses the macro **MAX_FOO**, then the objects **CFG_PARAMS** property must be set to **MAX_FOO**, and there must be a parameter object named **MAX_FOO** that defines the properties of the parameter. See *CFG_PARAMS* in Configuration Parameter Property on page 21 and Parameter Object and Properties on page 25.

For each source file identified by **CONFIGLETTES**, the project facility generates an #include directive for the file in the project's **prjConfig.c** file (as well as for all header files identified with **HDR_FILES**; see *HDR_FILES* below). The order of inclusion is not specified, and all files are compiled together with the code generated for the project. For more information about **prjConfig.c** and other project files, see About CDFs, VxWorks Configuration, and Build on page 12.

As all source files identified by **CONFIGLETTES** (those associated with included components) are compiled together. Be careful not to modify any of the configuration macro values in a source file, as the modification could be visible in other source files, with unpredictable results.

The default directory for source code identified by **CONFIGLETTES** is *layerDir* **/configlette**, where *layerDir* is the root directory of the component VxWorks layer. Header files for these code fragments are identified with the **HDR_FILES** property (see *HDR_FILES* below).

> Note: Do not modify the Wind River-supplied layer source files in *installDir* **/vxworks-7/pkgs_v2**. Create a copy of the layer and modify the copied layer files instead. For more information, see *VxWorks 7 Layers and Package Management Guide*.

HDR_FILES

Identifies the header files required for the component's initialization routine and other source code. For example:

```
HDR_FILES foo.h bar.h
```

The header files referenced by this property are located in *layerDir* **/h**, where *layerDir* is the root directory of the component VxWorks layer.

For each header file identified by **HDR_FILES**, the project facility generates an #include directive for the file in the project's **prjConfig.c** file (as well as for all source files identified with **CONFIGLETTES**; see *CONFIGLETTES* above). All files are compiled together with the code generated for the project. For more information about **prjConfig.c** and other project files, see About CDFs, VxWorks Configuration, and Build on page 12.

See *INIT_RTN* Component Initialization Properties on page 19 for identification of the component's initialization routine; see *CONFIGLETTES* above for identification of a component's source code elements.

Parent topic: Module Linkage Properties on page 16

Configuration Parameter Property

The **CFG_PARAMS** identifies configuration parameters associated with the component, typically a list of preprocessor macros that are exposed to users for alteration through Workbench or vxprj.

Each configuration parameter must also be described separately in its own parameter object; see Parameter Object and Properties on page 25.

For example, if a parameter is identified in the component object as follows:

WNDRVR

```
Component INCLUDE_TRBL {
    ...
    CFG_PARAMS MAX_TRBL
    ...
}
```

Then, there must also be a parameter object defining it as follows:

```
Parameter MAX_TRBL {
    NAME maximum trbl
    SYNOPSIS The maximum trbl permitted. Higher number for more.
    TYPE int
    DEFAULT 0x29A
}
```

Component parameters must be implemented in the code identified by **CONFIGLETTES** or **INIT_RTN**; otherwise they have no effect. See *CONFIGLETTES* in Source File Properties on page 20 and *INIT_RTN* in Component Initialization Properties on page 19.

Parent topic: Module Linkage Properties on page 16

Component Dependency Properties

Dependencies between components are resolved automatically by the project facility when they are based on symbolic references.

This is true as long as the **MODULES** attribute has been properly used, in which case all the components' modules are linked with the image and the component is added to the project; see *MODULES* in Module Linkage Properties on page 16.

When references are, however, based on function pointers or are made to routines provided in source code, the required component must be identified explicitly.

For example, if **INCLUDE_FOO** provides **foo.o**, and there are calls in **foo.o** to logMsg( ), then the build system will automatically determine that it needs to add the **INCLUDE_LOGGING** component to the configuration of VxWorks when **INCLUDE_FOO** has been added.

If, however, **INCLUDE_FOO** includes calls to barCreate( ), which is provided by **INCLUDE_BAR**, and these calls are made by way of function pointers (or if the code provided by either component is in source format), then the **INCLUDE_FOO** component must use the **REQUIRES** property to identify this dependency so that **INCLUDE_BAR** is added to the configuration whenever **INCLUDE_FOO** is added.

The following properties define dependency relationships between components that are not resolved by symbolic references.

REQUIRES

A list of other component(s) that the component needs in order to function, which would not otherwise be included based on symbolic references. In particular, **REQUIRES** must be used whenever the required component is provided in source format or its routines are referenced by function pointers.

For example, if component A identifies B with **REQUIRES**, but B does not have a reciprocal assignment (nor linkage), then the following behaviors hold true:

- If you add A to a project, B is added automatically.
- If you subsequently remove B, then A is removed automatically because it cannot function without B. Similarly, if you remove A, then B is removed automatically because it was added based on the **REQUIRES** relationship, and not explicitly by a user.
- If you add B to a project, A is not added automatically, because B does not require it.

- If you subsequently add A, its dependency on B is already fulfilled.
- If you then remove A, B remains part of the project, as it was explicitly added by a user.

INCLUDE_WHEN

Sets a dependency to automatically include the component if the components assigned to **INCLUDE_WHEN** are included. This property is largely a convenience for the user, and does not indicate a hard dependence of one component on others.

The following means, for example, that the component would automatically be included if **INCLUDE_FOO** and **INCLUDE_BAR** are included in the configuration.

```
INCLUDE_WHEN INCLUDE_FOO INCLUDE_BAR
```

The component can be removed from the configuration without affecting **INCLUDE_FOO** or **INCLUDE_BAR** presence; but if either **INCLUDE_FOO** or **INCLUDE_BAR** is removed, the component is removed as well.

Parent topic: Module Linkage Properties on page 16

Workbench Display Properties

The **NAME**, **SYNOPSIS**, **_CHILDREN**, and **_DEFAULTS** properties control the display of information about a component in Workbench.

NAME

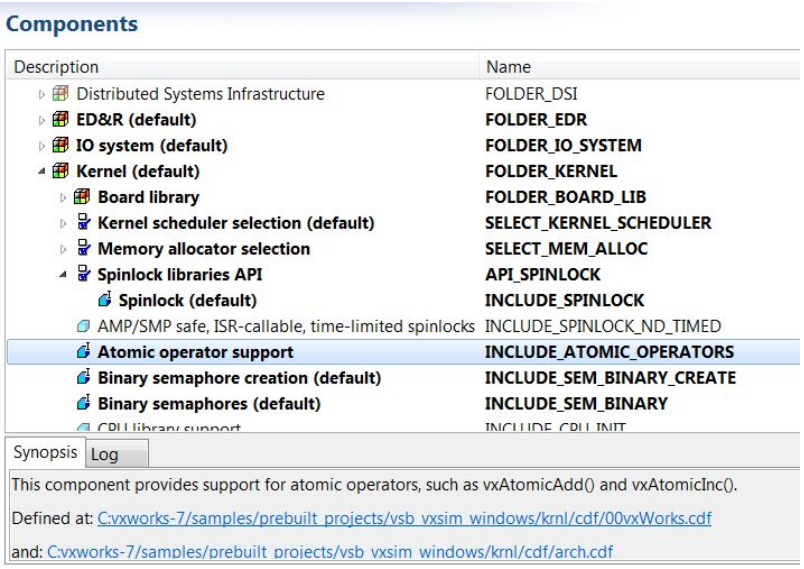
A descriptive string that appears in the Description column in the Workbench display of components as shown in the image below. This string is intended to provide a more user-friendly description than that provided in the Name column, which is the identifier used with the vxprj command-line configuration tool. This string must provide a brief description of the component (due to limitations of display space), whereas the **SYNOPSIS** property can be used to provide a fuller description.

Note that the component name is not the same as the more user-friendly descriptive string used with the **NAME** property. The component name (in the form **INCLUDE_FOO**) appears in the **Name** column in the Workbench display of components as shown in the image below. For more information about component names, see Component Name on page 15.

SYNOPSIS

A descriptive string that appears in the Synopsis tab in the Workbench **Components** window as shown in the image below. This string must give a fuller description than that provided by the component **NAME** attribute. Use this string to answer questions like, "Which API functions are provided by this component?" or "What specific function does this component contribute to the system?".

Workbench will report a live link to the component CDF files in the Synopsis tab below your descriptive string. Clicking the link opens the CDF file in the Workbench editor.
Figure 1. Workbench Component Display



Like **NAME** and **SYNOPSIS**, the following properties also affect user presentation in Workbench:

_CHILDREN

This component is a member of the specified folder or selection. This property allows for adding the component to a folder or selection without modifying the folder or selection object itself. If the folder or selection that is referenced does not exist, the component will not be displayed in Workbench.

See Workbench Folder Object and Properties on page 36 and Workbench Selection Object and Properties on page 38; also see CDL Underscore Operator and Object Relationships on page 35.

_DEFAULTS

This component is a default component of the specified folder or selection. This property allows for adding the component to a folder or selection without modifying the folder or selection object itself. It must be used in conjunction with **_CHILDREN**.

Also see CDL Underscore Operator and Object Relationships on page 35, *DEFAULTS* in Folder Properties on page 37 and Workbench Selection Object and Properties on page 38.

Parent topic: Module Linkage Properties on page 16

Component Example

As an example, a message logging component is defined in CDL.

Pre-VxWorks SR0600 release, the logging component is defined as follows:

```
Component INCLUDE_LOGGING {
        NAME            Message logging
        SYNOPSIS        This component provides the logLib library. This \
                        library handles message logging. It is usually used to \
                        display error messages on the system console, but such \
                        messages can also be sent to a disk file or printer. \
                        The logMsg() and logTask() functions are the basic \
                        components of the logging system. The logMsg() \
                        function has the same calling sequence as printf(), \
```


WNDRVR

```
                        but instead of formatting and outputting the message \
                        directly, it sends the format string and arguments to \
                        a message queue. The logTask() task waits for messages \
                        on this message queue. It formats each message \
                        according to the format string and arguments in the \
                        message, prepends the ID of the sender, and writes it \
                        on one or more file descriptors that have been \
                        specified as logging output streams (by logInit() \
                        or subsequently set by logFdSet() or logFdAdd()).
        MODULES         logLib.o
        INIT_RTN        logInit (consoleFd, MAX_LOG_MSGS, LOG_TASK_PRIORITY);
        PROTOTYPE       extern STATUS logInit (int, int, UINT);
        CFG_PARAMS      MAX_LOG_MSGS               \
                        LOG_TASK_PRIORITY
        HDR_FILES       logLib.h                   \
                        private/logLibP.h
        REQUIRES        INCLUDE_MSG_Q              \
                        INCLUDE_ANSI_STDIO         \
                        INCLUDE_SEM_MUTEX          \
                        INCLUDE_ANSI_STRNCPY       \
                        INCLUDE_TASK_CREATE_DELETE \
                        INCLUDE_MSG_Q_CREATE_DELETE\
                        INCLUDE_ATOMIC_OPERATORS
}
```

For illustrative purposes, in another configuration system using #ifdef/#endif statements, the definition of the logging component might look like this:

```
#ifdef INCLUDE_LOGGING
    logInit (consoleFd, MAX_LOG_MSGS, LOG_TASK_PRIORITY);
#endif /* INCLUDE_LOGGING */
```

Parent topic: Component Properties on page 16

Related information
Module Linkage Properties on page 16

3.3. Parameter Object and Properties

Parameter objects are used to define component configuration parameters that are presented to the user for modification by way of Workbench or vxprj.

Parameter objects must be defined in the same CDF with their associated component object. Configuration parameters must be associated with the appropriate component by way of the component object's **CFG_PARAMS** property.

The source code that uses the parameters must be identified with the component's **CONFIGLETTES** property. For information about these properties, see *CFG_PARAMS* in Configuration Parameter Property on page 21 and *CONFIGLETTES* in Source File Properties on page 20. For an example of how all of these definitions are used together, see Parameter Example on page 27.

Parameter Template

```
Parameter PARAMETER_MY_PARAM {
    NAME       user-friendly name for GUI
    SYNOPSIS   description for GUI
```

```
    TYPE      [ C type | exists | untyped ]
    DEFAULT  defaultValueForParam
}
```

Parameter Name on page 26 The parameter name is used by both Workbench and vxprj to identify the parameter.
Parameter Properties on page 26 The **NAME**, **TYPE**, and **DEFAULT** properties describe a parameter.
Parameter Example on page 27 The tip component object and its related parameter objects implement the **TIP_CONFIG_STRING** and **TIP_ESCAPE_CHARACTER** configuration parameters—that are used as arguments for the initialization routine.

Parent topic: Component Object and Properties on page 14

Parameter Name

The parameter name is used by both Workbench and vxprj to identify the parameter.

The parameter name must conform to the following rules:

- It must also be a valid C macro name.
- It must be assigned to the relevant component's object **CFG_PARAMS** property to associate the parameter with the component (see *CFG_PARAMS* in Configuration Parameter Property on page 21).
- It must not include a prefix used for other object type: **INCLUDE_**, **BUNDLE_**, **PROFILE_**, **FOLDER_**, and **SELECT_**. Naming is otherwise unrestricted. For example, you can use *FOO_XXX*, but not *INCLUDE_FOO*.

The syntax for parameter-name assignment is as follows, using the Parameter keyword and name before the brackets that enclose the parameter-object properties:

```
Parameter PARAMETER_NAME {
    // parameter properties go here
}
```

Also see Parameter Object and Properties on page 25 and Parameter Example on page 27 for the context in which the name assignment is made. The parameter name appears in the Name column in the Workbench display of components (see Workbench Display Properties on page 23).

Note that the parameter name is not the same as the more user-friendly descriptive string used with the **NAME** property

Parent topic: Parameter Object and Properties on page 25

Parameter Properties

The **NAME**, **TYPE**, and **DEFAULT** properties describe a parameter.

NAME

A descriptive string that appears in the Description column in the Workbench display of components (see Workbench Display Properties on page 23). This string is intended to provide a more user-friendly description than that provided in the Name column, which is the identifier used with the vxprj command-line configuration tool.

TYPE

Defines the data type of the parameter, which can be **int**, **uint**, **bool**, **string**, **exists**, or **untyped**.

WNDRVR

Must be used when the parameter name is defined or undefined as a C macro (the default can be set to either **TRUE** or **FALSE**). For example, the data type of the configuration parameter **CLEAR_BSS** is exists. If its value is set to **TRUE** (the default), the following appears in the **prjParams.h** project file:

```
#undef CLEAR_BSS
#define CLEAR_BSS
```

For more information about **prjParams.h** and other project files, see About CDFs, VxWorks Configuration, and Build on page 12.

DEFAULT

The default value of the parameter. If it is not set, the user must define a value in order to use the component.

Parent topic: Parameter Object and Properties on page 25

Parameter Example

The tip component object and its related parameter objects implement the **TIP_CONFIG_STRING** and **TIP_ESCAPE_CHARACTER** configuration parameters—that are used as arguments for the initialization routine.

```
Component INCLUDE_TIP {
    NAME         tip serial line connection utility
    SYNOPSIS     interactive utility to connect to and manage \
    multiple     serial lines
    CONFIGLETTES usrTip.c
    MODULES      tipLib.o
    INIT_RTN     usrTipInit (TIP_CONFIG_STRING, TIP_ESCAPE_CHARACTER);
    HDR_FILES    private/tipLibP.h
    LINK_SYMS    tipStart
    INCLUDE_WHEN INCLUDE_SHELL
    CFG_PARAMS   TIP_CONFIG_STRING\
                 TIP_ESCAPE_CHARACTER
    HELP         tip
}
Parameter TIP_CONFIG_STRING {
    NAME         Tip global configuration string
    SYNOPSIS     Used to define connection lines attributes
    TYPE         string
    DEFAULT      ""
}
Parameter TIP_ESCAPE_CHARACTER {
    NAME         Tip escape character
    SYNOPSIS     Character toggling command mode
    TYPE         string
    DEFAULT      "~"
}
```

Note that the parameters are used in the component's initialization routine usrTipInit( ), and are identified as the component's parameters with the **CFG_PARAMS** attribute; and that each is then defined with its own parameter object.

The prototype for usrTipInit( )—which is provided by the **usrTip.c** file that is identified by **CONFIGLETTES** in the component object—is as follows:

```
void usrTipInit
    (
    char * configString,
    char * escapeCharStr
    )
```

After building the project, the **prjConfig.c** file includes the following:

```
usrTipInit (TIP_CONFIG_STRING, TIP_ESCAPE_CHARACTER); /* interactive utility to
connect to and manage multiple serial lines */
```

And **prjParams.h** includes the following definitions (if the parameters have not been changed from the defaults):

```
#undef TIP_CONFIG_STRING
#define TIP_CONFIG_STRING ""
#undef TIP_ESCAPE_CHARACTER
#define TIP_ESCAPE_CHARACTER "~"
```

For more information about **prjConfig.c**, **prjParams.h**, and other project files, see About CDFs, VxWorks Configuration, and Build on page 12.

Parent topic: Parameter Object and Properties on page 25

3.4. Initialization Group Object and Properties

If a component has an initialization routine, it must be identified as such in the component object (using the component **INIT_RTN** attribute), and it must also be registered with an initialization group.

There are two alternatives for registering a component with an initialization group, and for controlling the order in which it is initialized within that group:

- From within the component object, use the **_INIT_ORDER** property to register the component with an initialization group, and (optionally) **INIT_BEFORE** or **INIT_AFTER** to control the order. Use this method to add a component to an existing initialization group without modifying the object for that group. For more information, see *_INIT_ORDER*, *INIT_BEFORE*, and *INIT_AFTER* in Component Initialization Properties on page 19.
- From within an initialization group object, use the **INIT_ORDER** property to both register the component with the group and to control the order of initialization. Use this method if you want to create a new initialization group for a group of custom components. For more information, see *INIT_ORDER*, *INIT_BEFORE*, and *INIT_AFTER*, in Initialization Group Properties on page 30.

The order in which the initialization groups themselves are processed is controlled by properties for the initialization group objects.

Initialization Group Objects on page 29 Initialization groups provide the means for logically grouping the initialization of related components, and sequencing the initialization of all system facilities at start-up.
Initialization Group Name on page 29 All initialization group names must use a prefix of init to avoid naming conflicts.
Initialization Group Properties on page 30 Several properties describe an initialization group.
Initialization Group Example on page 31 The **INCLUDE_FOO** sample component illustrates a simple initialization group example.

Parent topic: Component Object and Properties on page 14

WNDRVR

Initialization Group Objects

Initialization groups provide the means for logically grouping the initialization of related components, and sequencing the initialization of all system facilities at start-up.

Initialization groups must have their own initialization routine. They may be nested within another initialization group, or registered directly with the VxWorks root initialization group, **usrRoot**, which is the entry point for post-kernel initialization. If an initialization group is not registered directly with **usrRoot**, it must be registered in a sequence of initialization groups whose top-level initialization group is registered with **usrRoot**.

For example, the kernel shell has initialization routines for each component; these routines are registered with an initialization group for the shell, and that group is registered with **usrRoot**.

The code in an initialization group is synthesized by the project facility into the file **prjConfig.c**, which is generated for each project. For more information about **prjConfig.c** and other project files, see About CDFs, VxWorks Configuration, and Build on page 12

Note: Do not modify Wind River initialization group objects.

Initialization Group Template

```
InitGroup initMyGroup {
    SYNOPSIS     description of initialization group
    INIT_RTN     initMyGrpRtn();
    _INIT_ORDER [ usrRoot | customInitGroup ]
    INIT_ORDER  INCLUDE_FOO1 \
        INCLUDE_FOO2 \
        INCLUDE_FOO3 \
        ...
    INIT_BEFORE someOtherInitGroup
}
```

Parent topic: Initialization Group Object and Properties on page 28

Initialization Group Name

All initialization group names must use a prefix of init to avoid naming conflicts.

Wind River initialization groups use the prefix **usr** for backwards compatibility reasons.

The name must be unique. The syntax for initialization-group-name assignment is as follows, using the InitGroup keyword and name before the brackets that enclose the initialization-group-object properties:

```
InitGroup initMygroup {
    // initialization group property assignments go here
}
```

Also see Initialization Group Objects on page 29 and Initialization Group Example on page 31 for the context in which the name assignment is made.

Parent topic: Initialization Group Object and Properties on page 28

Initialization Group Properties

Several properties describe an initialization group.

NAME

The **NAME** property can be used to provide a short name for the initialization group; however, it does not appear anywhere in the Workbench user interface.

SYNOPSIS

The **SYNOPSIS** property is used to provide a brief, readable definition of the initialization group.

INIT_RTN

The initialization routine for the initialization group itself. This property could be a valid piece of C code, or a function call, including a semicolon at the end. You could use this property to set the value of a global variable, or you could call a function within the group of components.

If you call a function, this can be emplemented either in a component configlette, or in the component's library code.

When this routine is called at system startup, the initialization routines for all of the components assigned to the initialization group are subsequently called. (Components can be assigned to an initialization group with either the initialization group **INIT_ORDER** property or the component property **_INIT_ORDER**; see *_INIT_ORDER* in Component Initialization Properties on page 19 and Parameter Properties on page 26).

_INIT_ORDER

Assigns the initialization group to the specified initialization group. This property is used to add the initialization group to an existing initialization group. The alternative is to add the initialization group directly to another initialization group's object with **INIT_ORDER** (see *INIT_ORDER* in Parameter Properties on page 26 and CDL Underscore Operator and Object Relationships on page 35). In most cases, custom initialization groups are initialized after VxWorks initialization groups, which is specified as follows:

```
_INIT_ORDER usrRoot
```

Any sequence of custom initialization groups must ultimately be registered with usrRoot. That is, if an initialization group is not registered directly with usrRoot, it must be registered in a sequence of initialization groups whose top-level initialization group is registered with usrRoot.

An initialization group that is bound to an existing initialization group using the **_INIT_ORDER** property is, by default, initialized last within that group. The **INIT_BEFORE** and **INIT_AFTER** properties can be used to change the order.

INIT_ORDER

Components belonging to this initialization group listed in the order in which they must be initialized. Each of the components listed must have its own initialization routine identified with the component **INIT_RTN** property.

See *INIT_RTN* in Component Initialization Properties on page 19. Other initialization groups can also be listed in an initialization group's **INIT_ORDER** property to control sequencing of custom initialization groups.

INIT_BEFORE

Calls the initialization routine of this initialization group before the initialization group specified by this property. This property can be used to define precisely where in the initialization sequence the initialization group is placed.

INIT_AFTER

Call the initialization routine of this initialization group after the one specified with this property. This property can be used to specify precisely where the component is placed in the initialization sequence of the initialization group specified by **_INIT_ORDER**.

WNDRVR

Parent topic: Initialization Group Object and Properties on page 28

Initialization Group Example

The **INCLUDE_FOO** sample component illustrates a simple initialization group example.

Assume, for example, that the **INCLUDE_FOO** component includes the following definition of its initialization routine:

```
Component INCLUDE_FOO {
...
    INIT_RTN    fooInit(arg1, arg2);
    _INIT_ORDER initFooBar
    INIT_BEFORE INCLUDE_BAR
...
}
```

The initialization group object would look something like this:

```
InitGroup initFooBar {
    INIT_RTN    fooBarInit();
    SYNOPSIS    initialization group for foobar
    _INIT_ORDER usrRoot
    INIT_ORDER  INCLUDE_ABC \
                INCLUDE_BAR \
                INCLUDE_BAR2
}
```

In this example, the **INCLUDE_FOO** component is registered with the **initFooBar** initialization group. The components in that group are initialized after those in the VxWorks **usrRoot** initialization group, in the following order:

1. **INCLUDE_ABC**
2. **INCLUDE_FOO**
3. **INCLUDE_BAR**
4. **INCLUDE_BAR2**

Parent topic: Initialization Group Object and Properties on page 28

3.5. Bundle Object and Properties

Bundle objects can be used to associate components that are often used together, which facilitates configuration of the operating system with useful sets of facilities.

For example, **BUNDLE_POSIX** provides all the components required for POSIX support in the kernel.

A bundle can be added and removed from a VxWorks image project (VIP). The project facility does not, however, track the addition or removal of a bundle as such, rather it tracks the set of components it identifies. When a bundle is used to add components, the facility records the components for the project, but nothing about the bundle. Removing a bundle does not force all the components out, but simply removes the components that are not otherwise required by the system.

Bundle Template

```
Bundle BUNDLE_MY_BUNDLE {
    NAME        user-friendly name for GUI
    SYNOPSIS    description for GUI
    COMPONENTS INCLUDE_FOO1 \
               INCLUDE_FOO2 \
               INCLUDE_FOO3 \
               ...\
}
```

Bundle Name on page 32 The bundle name is used by both Workbench and vxprj to identify the bundle.
Bundle Properties on page 32 Bundles support the **NAME**, **SYNOPSIS** and **COMPONENTS** properties.
Bundle Example on page 33 The network kernel shell (**BUNDLE_NET_SHELL**) includes a related set of components, such as the kernel shell, loader, network symbol table, and so on.

Parent topic: Component Object and Properties on page 14

Bundle Name

The bundle name is used by both Workbench and vxprj to identify the bundle.

The bundle name is used by both Workbench and vxprj to identify the bundle. The name must be unique, and a **BUNDLE_** prefix is recommended to avoid naming conflicts. The syntax for bundle-name assignment is as follows, using the Bundle keyword and name before the brackets that enclose the bundle-object properties:

```
Bundle BUNDLE_MY_BUNDLE {
    // bundle property assignments go here
}
```

Also see Bundle Object and Properties on page 31 and Bundle Example on page 33 for the context in which the name assignment is made.

Note that the bundle name is not the same as the more user-friendly descriptive string used with the **NAME** property.

Parent topic: Bundle Object and Properties on page 31

Bundle Properties

Bundles support the **NAME**, **SYNOPSIS** and **COMPONENTS** properties.

NAME

A descriptive string. This string is intended to provide a more user-friendly description than that provided in the Name column, which is the identifier used with the vxprj command-line configuration tool.

SYNOPSIS

A descriptive string. Use this string to provide a fuller description than that provided by the **NAME** attribute.

COMPONENTS

A list of components to add to the kernel configuration when this bundle is added.

Parent topic: Bundle Object and Properties on page 31

Bundle Example

The network kernel shell (**BUNDLE_NET_SHELL**) includes a related set of components, such as the kernel shell, loader, network symbol table, and so on.

```
Bundle BUNDLE_NET_SHELL {
    NAME        Network kernel shell
    SYNOPSIS    This bundle provides kernel shell tools with \
                a networking symbol table.
    HELP        shell
    COMPONENTS  INCLUDE_NET_SYM_TBL \
                INCLUDE_SHELL \
                INCLUDE_SHELL_EMACS_MODE \
                INCLUDE_SHELL_VI_MODE \
                INCLUDE_SHELL_HISTORY_FILE \
                INCLUDE_LOADER \
                INCLUDE_DISK_UTIL \
                INCLUDE_SHOW_ROUTINES \
                INCLUDE_STAT_SYM_TBL \
                INCLUDE_DEBUG \
                INCLUDE_UNLOADER \
                INCLUDE_MEM_SHOW \
                INCLUDE_SYM_TBL_SHOW \
                INCLUDE_CPLUS
}
```

For information about components, see Component Object and Properties on page 14.

Parent topic: Bundle Object and Properties on page 31

3.6. Profile Object and Properties

A profile object defines a set of components to be used as a starting point to create a project.

For example, **PROFILE_DEVELOPMENT** provides a VxWorks system that includes development and debugging components, and support for real-time processes.

The profile can only be used when a VxWorks image project (VIP) is created. When used, the profile replaces the default VxWorks configuration. Components defined by the profile are added to the project as if they were added by the user. Therefore, after creation of the project, there is no trace of the profile in the project. A profile, as such, cannot be removed. Profiles can be built on other profiles.

Profile Template

```
Profile PROFILE_NAME {
    NAME        user-friendly name for GUI
    SYNOPSIS    description for GUI
    PROFILES    PROFILE_A ...
    COMPONENTS  INCLUDE_FOO1 \
                INCLUDE_FOO2 \
                INCLUDE_FOO3 \
```


WNDRVR

```
                    . . .
}
```

Profile Name on page 34 The profile name is used by both Workbench and vxprj to identify the profile.
Profile Properties on page 34 Profiles support the **NAME**, **SYNOPSIS**, **PROFILES**, and **COMPONENTS** properties.
Profile Example on page 35

Parent topic: Component Object and Properties on page 14

Profile Name

The profile name is used by both Workbench and vxprj to identify the profile.

The name must be unique and a **PROFILE_** prefix is recommended to avoid naming conflicts.

The syntax for profile-name assignment is as follows, using the Profile keyword and name before the brackets that enclose the profile-object properties:

```
Profile PROFILE_NAME {
    // profile property assignments go here
}
```

Also see Profile Object and Properties on page 33 for the context in which the name assignment is made.

The profile name appears in the Workbench VIP Configuration Profile dialog. Note that the profile name is not the same as the more user-friendly descriptive string used with the **NAME** property.

Parent topic: Profile Object and Properties on page 33

Profile Properties

Profiles support the **NAME**, **SYNOPSIS**, **PROFILES**, and **COMPONENTS** properties.

NAME

A descriptive string that appears in the Workbench VIP Configuration Profile dialog. This string is intended to provide a more user-friendly description than that provided in the Name column, which is the identifier used with the vxprj command-line configuration tool.

SYNOPSIS

A descriptive string that appears in the Workbench VIP Configuration Profile dialog. Use this string to provide a fuller description than that provided by the **NAME** attribute.

PROFILES

A list of base profiles.

COMPONENTS

A list of components to add to the kernel configuration when this profile is added.

Parent topic: Profile Object and Properties on page 33

WNDRVR

Profile Example

These examples are truncated because they include so many components.

Before the VxWorks SR0600 release, refer to the kernel layer. See *installDir* **/vxworks-7/pkgs/os/core/kernel-***a.b.c.d* **/cdf/01profile_development.cdf**, where *a.b.c.d* is the current version of the VxWorks kernel layer.

```
Profile PROFILE_DEVELOPMENT
{
    NAME            VxWorks kernel development configuration profile
    SYNOPSIS    VxWorks kernel including development and debugging components. \
                The VxWorks Debug Agent will be included in your system.  This \
                agent does not provide a secured network connection; it is advised \
                to not include it in your development system or make sure it is \
                not automatically started at boot time through boot line parameter \
                or dedicated component.
    PROFILES    BSP_DEFAULT
#if (defined _WRS_CONFIG_SMP) || (defined _WRS_CONFIG_LP64)
    COMPONENTS  INCLUDE_ANSI_ASSERT \
                INCLUDE_ANSI_CTYPE \
                INCLUDE_ANSI_LOCALE \
                INCLUDE_ANSI_MATH \
                INCLUDE_ANSI_STDIO \
                INCLUDE_ANSI_STDIO_EXTRA \
                INCLUDE_ANSI_STDLIB \
                INCLUDE_ANSI_STRING \
                INCLUDE_ANSI_TIME \
                INCLUDE_APPL_LOG_UTIL \
                INCLUDE_BASE_KERNEL \
                INCLUDE_BOOT_LINE_INIT \
```

With VxWorks SR0600 and later releases, the kernel is no longer a layer.

Parent topic: Profile Object and Properties on page 33

3.7. CDL Underscore Operator and Object Relationships

The underscore operator can be used with object properties that define a relationship to reverse the definition of that relationship—usually along the lines of changing list-of-members to member-of.

For example, if FOREST were a property of a landscape object used to identify its trees, then _FOREST could be used in a tree object to identify it as belonging to a particular landscape, without having to modify the landscape object itself.

The most common uses of the underscore operator are to assign a new component object to an existing initialization group or Workbench folder—**using _INIT_ORDER** and **_CHILDREN** respectively—without having to modify the existing object by adding the name of the new component to its list.

For example, assume that the CDF for folder **FOLDER_EXISTING** includes the following:

```
Folder FOLDER_EXISTING {
    NAME The existing folder
    SYNOPSIS The folder that’s been around a while
    CHILDREN INCLUDE_FOO \
    ...
}
```

Then a new component **INCLUDE_BAR** can be added to the Workbench folder without modifying the folder CDF by using **_CHILDREN** in the component CDF, as follows:

```
Component INCLUDE_BAR
    ...
    _CHILDREN FOLDER_EXISTING
    ...
}
```

Note that a relationship can be defined using forward or reverse declaration, but there are subtle differences. The forward declaration defines a relationship. Any subsequent forward declaration overrides all previous ones (forward or reverse). The reverse declaration is typically used to add to an existing relationship, and all reverse declaration are cumulative.

Parent topic: Component Object and Properties on page 14

3.8. Workbench Folder Object and Properties

Folder objects control the display of components in the Workbench kernel configuration facility.

They create a directory-type hierarchy for grouping components that are logically related, and present useful information about each component, such as:

- Its association with other components.
- Its configuration parameters and their default settings.
- Whether or not it is included in the current project.
- Whether or not it is available in the current installation.

Folders can contain components, selections, and other folders.

Folders themselves serve as configuration elements—their contents can be selected as a unit for inclusion or removal from a VxWorks image project (or a sub-set of the contents, depending on what is identified in the folder object as defaults). Folders are therefore unlike bundles and profiles, which can only be used to include a set of components as a unit. Folders also present configuration parameters for modification.

The **FOLDER_ROOT** folder is the base of the folder hierarchy. Any sequence of folders must ultimately be registered with **FOLDER_ROOT**. That is, if a folder is not registered directly with **FOLDER_ROOT**, it must be registered in a sequence of folders whose top-level folder is registered with **FOLDER_ROOT**.

Note: Use folder objects only when creating a new grouping of components (new or existing). Do not modify existing folders in order to include new components (except by way of the **_CHILDREN** property).

Folder Template

```
Folder FOLDER_NAME {
    NAME     user-friendly name for GUI
    SYNOPSIS description for GUI
    CHILDREN INCLUDE_FOO1 \
             INCLUDE_FOO2 \
             INCLUDE_FOO3 \
             ...
    DEFAULTS INCLUDE_FOO3 \
    ...
}
```

Folder Name on page 37 The folder name must be unique and must also have a **FOLDER_** prefix to avoid naming conflicts.
Folder Properties on page 37 The **NAME**, **SYNOPSIS**, **CHILDREN**, **_CHILDREN**, and **DEFAULTS** properties describe a folder:
Folder Example on page 38 Folders can contain more than one component.

Parent topic: Component Object and Properties on page 14

Folder Name

The folder name must be unique and must also have a **FOLDER_** prefix to avoid naming conflicts.

The syntax for folder-name assignment is as follows, using the Folder keyword and name before the brackets that enclose the folder-object properties:

```
Folder FOLDER_MY_FOLDER {
    // folder property assignments go here
}
```

Also see Workbench Folder Object and Properties on page 36 and Folder Example on page 38 for the context in which the name assignment is made.

The folder name appears in the Name column in the Workbench display of components (see Workbench Display Properties on page 23). Note that the folder name is not the same as the more user-friendly descriptive string used with the **NAME** property.

Parent topic: Workbench Folder Object and Properties on page 36

Folder Properties

The **NAME**, **SYNOPSIS**, **CHILDREN**, **_CHILDREN**, and **DEFAULTS** properties describe a folder:

NAME

A descriptive string that appears in the Description column in the Workbench display next to the folder icon.

SYNOPSIS

A descriptive string that appears in the Synopsis tab in the Workbench display of components, profiles, and so on. Use this string to provide a fuller description than that provided by the **NAME** attribute.

CHILDREN

Identifies the components, folders, and selections belonging to this folder.

_CHILDREN

This folder is a child folder of the specified folder— it must exist in the folder hierarchy. This property is used to add a folder to another folder, without modifying the folder to which it is added. For more information, see CDL Underscore Operator and Object Relationships on page 35.

The **FOLDER_ROOT** folder is the base of the folder hierarchy. All folders must be registered with it directly or through other folders in order to be displayed in Workbench.

DEFAULTS

The default component(s) that would be included if the folder itself were added to the project without any overrides. Folder groupings can affect configuration when a folder is added, because components specified by a folder's **DEFAULTS** property are added all at once. The **DEFAULTS** property is useful for complex sub-systems because it provides a shortcut for configuring the system with a useful, predetermined subset of selections.

Parent topic: Workbench Folder Object and Properties on page 36

Folder Example

Folders can contain more than one component.

For example, the ANSI functionality contained in a folder might be described as follows:

```
Folder FOLDER_ANSI {
       NAME            ANSI C components (libc)
       SYNOPSIS        This folder includes components and parameters to \
                       support the ANSI libraries.
       CHILDREN        INCLUDE_ANSI_ASSERT     \
                       INCLUDE_ANSI_CTYPE      \
                       INCLUDE_ANSI_LOCALE     \
                       INCLUDE_ANSI_MATH       \
                       INCLUDE_ANSI_STDIO      \
                       INCLUDE_ANSI_STDLIB     \
                       INCLUDE_ANSI_STRING     \
                       INCLUDE_ANSI_STRDUP     \
                       INCLUDE_ANSI_STRERROR   \
                       INCLUDE_ANSI_TIME       \
                       INCLUDE_ANSI_STDIO_EXTRA
       DEFAULTS        INCLUDE_ANSI_ASSERT     \
                       INCLUDE_ANSI_CTYPE      \
                       INCLUDE_ANSI_MATH       \
                       INCLUDE_ANSI_STDIO      \
                       INCLUDE_ANSI_STDLIB     \
                       INCLUDE_ANSI_STRING     \
                       INCLUDE_ANSI_TIME       \
                       INCLUDE_ANSI_STRDUP     \
                       INCLUDE_ANSI_STRERROR
}
```

Parent topic: Workbench Folder Object and Properties on page 36

3.9. Workbench Selection Object and Properties

A selection object is used to display alternative options for components that provide related functionality (such as timestamp drivers) in the Workbench component configuration interface.

The selection object provides for identifying these options, and provides a count property that specifies whether only one component, some specific number greater than one component, or any number greater than one component, may be selected.

Selections may only operate on components; they cannot be used with other CDF objects (parameters, folders, BSPs, and so on).

Selection Template

```
Selection SELECT_MY_SELECTION {
    NAME      user-friendly name for GUI
    SYNOPSIS  description for GUI
    COUNT     min-max
    CHILDREN  INCLUDE_FOO1 \
              INCLUDE_FOO2 \
```

```
            INCLUDE_FOO3 \
            ...
    DEFAULTS INCLUDE_FOO3 \
    ...
}
```

Selection Name on page 39 The selection name must be unique, and must also have a **SELECT_** prefix to avoid name conflicts.
Selection Properties on page 39 The **NAME**, **SYNOPSIS**, **COUNT**, **CHILDREN**, and **DEFAULTS** properties describe a selection:
Selection Example on page 40 Review a selection from a set of timestamp drivers:

Parent topic: Component Object and Properties on page 14

Selection Name

The selection name must be unique, and must also have a **SELECT_** prefix to avoid name conflicts.

The syntax for selection-name assignment is as follows, using the Selection keyword and name before the brackets that enclose the selection-object properties:

```
Selection SELECT_MY_SELECTION {
    // folder property assignments go here
}
```

Also see Workbench Selection Object and Properties on page 38 and Selection Example on page 40 for the context in which the name assignment is made. The selection name appears in the Name column in the Workbench display of components (see Workbench Display Properties on page 23). Note that the folder name is not the same as the more user-friendly descriptive string used with the **NAME** property.

Parent topic: Workbench Selection Object and Properties on page 38

Selection Properties

The **NAME**, **SYNOPSIS**, **COUNT**, **CHILDREN**, and **DEFAULTS** properties describe a selection:

NAME

A descriptive string that appears in the Description column in the Workbench next to the selection icon.

SYNOPSIS

A descriptive string that appears in the Synopsis tab in the Workbench display of components, profiles, and so on. Use this string to provide a fuller description than that provided by the **NAME** attribute.

COUNT

Set a minimum and maximum count for the components that may be selected from available options identified with **CHILDREN**. Use 1-1 for exactly one; and use 1- for one or more.

CHILDREN

Components from which to make selection.

DEFAULTS

The default component(s). The number of components defined as **DEFAULT** must be consistent with the **COUNT** value.

WNDRVR

Parent topic: Workbench Selection Object and Properties on page 38

Selection Example

Review a selection from a set of timestamp drivers:

```
Selection       SELECT_TIMESTAMP {
        NAME         Select timestamping
        SYNOPSIS     Choose between sequential timestamping, user-defined timestamping \
                     and system-defined timestamping.
        COUNT        1-1
        CHILDREN     INCLUDE_SYS_TIMESTAMP      \
                     INCLUDE_USER_TIMESTAMP     \
                     INCLUDE_SEQ_TIMESTAMP
        DEFAULTS     INCLUDE_SEQ_TIMESTAMP
}
```

There are three timestamp drivers available, as indicated by the three values for the **CHILDREN** property. The **COUNT** permits a choice of one, that is, a minimum and a maximum of 1.

Parent topic: Workbench Selection Object and Properties on page 38

3.10. BSP Object and Properties

BSP objects define BSPs and their properties so that they can be selected for a VxWorks image project (VIP) using Workbench or the vxprj command-line project facility.

For information about BSP CDF file-naming requirements, see Component Definitions and CDF File Names on page 5.

For information about BSP development, see the *VxWorks 7 BSP and Driver Guide*.

BSP Template

```
Bsp bspName {
    NAME      descriptive string
    CPU       cpuName
    REQUIRES INCLUDE_FOO1 \
             INCLUDE_FOO2 \
             INCLUDE_FOO3 \
             ...
    FP [ hard ] [ soft ] [ ... ]
    ENDIAN [ big | little ]
    MP_OPTIONS [ UP | SMP | UP SMP | SMP_ONLY ]
    DATA_MODEL [ ILP32 | LP64 | ILP32 LP64 ]
    GUEST_OS_OPTION NOT_SUPPORTED
}
```

Parent topic: Component Object and Properties on page 14

3.11. BSP Name

A BSP must have a unique name. The BSP name is used by both Workbench and vxprj to identify the BSP.

WNDRVR

The name must be unique and must match the name of the directory in which it is installed (see CDF Processing in Layers on page 6). The syntax for BSP-name assignment is as follows, using the Bsp keyword and name before the brackets that enclose the BSP-object properties:

```
 Bsp myBspName {
    // BSP property assignments go here
}
```

Also see BSP Object and Properties on page 40 and BSP Example on page 42 for the context in which the name assignment is made.

The BSP name appears in Workbench display of BSPs, for example, in the Project Setup dialog.

For information about parameters, which can be included in BSP CDFs, see Parameter Object and Properties on page 25.

Parent topic: Component Object and Properties on page 14

3.12. BSP Properties

NAME

A descriptive string.

CPU

The name of the supported CPU.

REQUIRES

A list of component(s) that the BSP depends on, which must be included whenever a VxWorks configuration is based on that BSP. Not all BSPs require components.

FP

The list of floating-point operation modes. The BSP need only define floating-point modes when it supports a subset of the modes provided by the CPU. The first listed is taken as the default for a VSB project. For information about the modes supported for a given CPU, see the *VxWorks 7 Architecture Supplement*.

ENDIAN

Defines the endian mode. Values may be big and little. The BSP need only define an endian value when it supports a subset of the modes provided by the CPU.

MP_OPTIONS

Defines multiprocessing support. This property must not be left blank. Supported values are UP, SMP and SMP_ONLY. Use the SMP or SMP_ONLY value when the BSP supports symmetric multiprocessing (SMP) and UP to denote uniprocessor.

If a hybrid of UP and SMP is possible with this BSP, select UP SMP. If a hybrid configuration is not possible on an SMP capable BSP, select SMP_ONLY.

GUEST_OS_OPTION

Not supported.

DATA_MODEL

Defines whether the VxWorks image project (VIP) produced with this BSP supports a 32-bit or 64-bit data model. Some targets support both data models, though not at the same time. For those targets, both data models must be specified in this property. For details about the data model supported for a given CPU, see the *VxWorks 7 Architecture Supplement*.

WNDRVR

BUILD_SPECS

Defines which build specs are created for a VIP based on this BSP. For more information, refer to the *VxWorks 7 Configuration and Build Guide*.

BUILD_TARGETS

Defines which build targets are created for a VIP based on this BSP. For more information, refer to *VxWorks 7 Configuration and Build Guide*.

Parent topic: Component Object and Properties on page 14

3.13. BSP Example

Review code for a BSP object.

```
Bsp itl_arria10 {
    NAME                board support package
    CPU                 ARMARCH7
    ENDIAN              little
    MP_OPTIONS          UP SMP
    DATA_MODEL          ILP32
    BUILD_SPECS         default
    BUILD_TARGETS       uVxWorks vxWorks vxWorks.bin
    FP                  hard soft
    REQUIRES            INCLUDE_KERNEL                  \
                        INCLUDE_CPU_INIT                \
                        INCLUDE_MEM_DESC                \
                        INCLUDE_BOARD_LIB_INIT          \
                        DRV_BUS_FDT_SIMPLE              \
                        INCLUDE_EARLY_MMU               \
                        DRV_INTCTLR_FDT_ARM_GIC         \
                        DRV_TIMER_FDT_SYNOPSYS_DW_TIMER \
                        DRV_SIO_FDT_NS16550             \
                        DRV_CLK_FDT_ALT_SOC_A10         \
                        DRV_RSTMGR_FDT_ALT_SOC_A10      \
                        DRV_SYSMGR_FDT_ALT_SOC_A10      \
                        BOARD_SELECTION
}
```

Parent topic: Component Object and Properties on page 14